

Scheduling of a Flexible Flow Line with Limited Buffers and Bypass

Chapter 16.3 — Pinedo, *Scheduling: Theory, Algorithms, and Systems*

June 27, 2026

Abstract

This report analyzes the scheduling problem described in Section 16.3 of Pinedo’s *Scheduling: Theory, Algorithms, and Systems* [4]: the cyclic scheduling of a flexible flow line with limited buffers and job bypass, originally addressed at IBM via the Flexible Flow Line Loading (FFLL) algorithm [2]. We (1) formalize the problem and its Graham three-field classification, (2) derive and critically assess a mixed-integer programming (MIP) formulation with a fully linear, time-indexed buffer constraint and an explicit machine-eligibility constraint, (3) describe the three-phase FFLL heuristic, apply it to three original numerical examples – including one demonstrating machine eligibility restrictions – and compare it against both the unweighted and bottleneck-weighted variants of the Profile Fitting (PF) heuristic of McCormick et al. [3], adapted to the finite-buffer setting, and (4) propose an Iterated Greedy with Local Search (IG-LS) algorithm inspired by the iterated greedy framework of Ruiz and Stützle [5], implement it, and benchmark it against FFLL on ten randomly generated instances. Weighting the bottleneck machine improves PF’s average gap from 50.1% to 43.1%, mirroring Pinedo’s own example; FFLL nonetheless outperforms both PF variants on average (33.8% mean gap), while IG-LS further reduces FFLL’s average optimality gap to 15.1%, reaching the lower bound on two of the ten instances.

Contents

1	Problem Statement and Graham Classification	3
1.1	Problem Description	3
1.2	Objectives and Alternative Objective Functions	3
1.3	Graham Notation	4
2	MIP Formulation and Critical Assessment	5
2.1	Indices, Parameters and Decision Variables	5
2.2	MIP Model	6
2.3	Model Size and Complexity	7
2.4	Critical Assessment of the MIP	7
3	The FFLL Algorithm	8
3.1	Algorithm Description	8
3.2	Three Original Application Examples	9
3.3	The Profile Fitting (PF) Heuristic and Comparison with FFLL	11
3.4	Critical Assessment of the FFLL Heuristic	13

4	Improved Algorithm: Iterated Greedy with Local Search (IG-LS)	14
4.1	Design Rationale	14
4.2	Pseudocode	15
4.3	Experimental Results on Ten Random Instances	15
4.4	Critical Assessment of IG-LS	15

1 Problem Statement and Graham Classification

1.1 Problem Description

The problem considers a **flexible flow line (FFL)** – a production environment consisting of multiple stages arranged in series, where each stage contains one or more machines operating in parallel. The specific context in Section 16.3 of Pinedo [4] is the assembly of printed circuit boards (PCBs), an environment for which the Flexible Flow Line Loading (FFLL) algorithm was originally designed at IBM [2].

The environment has the following defining characteristics:

Jobs Each job represents a batch of relatively small identical items (e.g., PCBs). A set of jobs that together represent one full day’s production mix is called a *Minimum Part Set* (MPS). The manufacturing process is repetitive, so the MPS cycles through the system continuously.

Processing Each job must be processed at every stage on exactly one machine. Not all machines at a stage are necessarily identical – some may handle only specific job types.

Bypass A job may not require processing at every stage. If a job skips a stage, the material handling system transports it past that stage. This is modeled by setting the processing time of that job at the bypassed stage to zero, $p_{ij} = 0$.

Limited buffers Between stages, buffer space is finite. If a buffer is full, the material handling system halts – or, if recirculation is possible, the job bypasses the stage and recirculates.

Cyclic schedule Because production is repetitive, the objective is a *cyclic schedule*: the best ordering and timing of jobs within one MPS cycle, repeated indefinitely.

1.2 Objectives and Alternative Objective Functions

The FFLL algorithm pursues two objectives simultaneously [4]:

- **Primary objective – maximize throughput.** In cyclic scheduling this is equivalent to minimizing the cycle time of one MPS. The cycle time cannot be smaller than the workload of the bottleneck machine, so the algorithm seeks to approach this lower bound.
- **Secondary objective – minimize work-in-process (WIP).** Since buffer space is limited, reducing WIP lowers the probability of buffer overflow and the resulting blocking.

Alternative objective functions that could be substituted for $C_{\max}$ include:

- Minimizing weighted tardiness $\sum w_j T_j$ – relevant when jobs carry due dates and differing priorities.
- Minimizing total/average flow time $\sum C_j$ – reducing the average time a job spends in the system.
- Minimizing the number of tardy jobs $\sum U_j$ – relevant in customer-facing settings.
- Minimizing maximum lateness $L_{\max}$ – relevant when due-date violations carry hard consequences.

1.3 Graham Notation

We use the standard three-field notation $\alpha \mid \beta \mid \gamma$, presented by Pinedo [4] as an adapted version of the classical framework, together with Pinedo’s own machine-environment symbol for this specific class of problems. Pinedo denotes the flexible flow shop by **FFc**, where c is the number of stages in series, each stage being a bank of *parallel machines* [4]. The classification is therefore:

$$FFc \mid M_j, b_i < \infty \mid C_{\max}$$

Field	Symbol	Meaning
α (machine environment)	FFc	Flexible flow shop: c stages in series, each stage a bank of parallel machines [4]
β (constraints)	M_j	Machine eligibility restrictions: job j may only be processed on a subset M_j of the machines at its stage [4]
	$b_i < \infty$	Finite buffer capacity between stages [3]
γ (objective)	$C_{\max}$	Minimize makespan / cycle time of the MPS

Table 1: Graham three-field classification of the flexible flow line scheduling problem.

On omitting an informal “bypass” field entry. Bypass is not a standard Graham symbol in the first place, and the two genuine structural features an informal “*bypass*” term might seem to require are already covered by entries that *are* standard. A job that bypasses a stage is, in our data, simply a job with $p_{ij} = 0$ for every machine i at that stage – a property of the input data p_{ij} , not an additional constraint requiring its own β -symbol (Pinedo’s own chapter never introduces one either). Once machine eligibility is made explicit via the standard symbol M_j [4], where in the most general case M_j may be an arbitrary subset of the stage’s machines, the routing flexibility that an informal “bypass” notation would gesture at is already subsumed: M_j together with the convention $p_{ij} = 0$ for bypassed stages fully describes which machines (if any) a job visits at a given stage, without requiring a separate notation symbol. The classification above is therefore both more compact and more faithful to standard usage.

On the buffer-capacity entry. The entry $b_i < \infty$ is kept exactly as a finite-buffer-capacity statement, the most direct reading of the problem in Section 16.3. We attribute it to McCormick et al. [3] rather than introducing a separate buffer-specific notation symbol, since that paper – already cited in Section 3.3 for the Profile Fitting heuristic – establishes the key structural fact connecting limited-buffer and zero-buffer (blocking) flow shops: *any* flow shop with a limited buffer of capacity b between two stages can be represented exactly by inserting b zero-processing-time “buffer machines” between them, reducing the limited-buffer case to the zero-buffer (blocking) case studied extensively in the flow-shop literature. This is the same reduction that justifies our finite-buffer adaptation of Profile Fitting in Section 3.3, so citing it here keeps the notation, the buffer model, and the PF heuristic discussion all anchored to a single source rather than introducing a new one solely for this purpose.

Remark. FFc generalizes both the classical flow shop (Fm , single machine per stage) and the parallel-machine environment (Pm , a single stage); it reduces to Fm when every stage has exactly one machine. Machine eligibility restrictions (M_j) mean routing is no longer uniform across jobs – a job may be restricted to a strict subset of the machines at a stage it visits – and finite buffers ($b_i < \infty$) introduce *blocking*, a phenomenon absent from most classical scheduling theory. Even the two-machine flow shop with blocking is known to be NP-hard [1], and the combination of parallel machines, machine eligibility, and finite buffers studied here is at least as hard. This NP-hardness is the central motivation for the heuristic (rather than exact) solution approach of Section 3.

2 MIP Formulation and Critical Assessment

Pinedo's chapter does not present an explicit mixed-integer program; we construct one here that mirrors the three decisions made sequentially by the FFL algorithm: machine assignment, job sequencing, and timing.

2.1 Indices, Parameters and Decision Variables

Indices and parameters.

- n : number of jobs in the MPS, indexed $j = 1, \dots, n$.
- m : total number of machines, indexed $i = 1, \dots, m$.
- S : number of stages, indexed $s = 1, \dots, S$.
- M_s : set of machines belonging to stage s .
- p_{ij} : processing time of job j on machine i (0 if job j bypasses the stage containing machine i).
- $E_{ij} \in \{0, 1\}$: *eligibility* indicator, $E_{ij} = 1$ if machine i is capable of processing job j and 0 otherwise. This captures the fact, noted explicitly in the problem description, that not all machines within a stage are identical and a given job may be restricted to a specific subset of machines at that stage.
- b_s : buffer capacity between stage s and stage $s + 1$.
- H : a finite planning horizon long enough to contain any feasible schedule of one MPS, e.g. $H = \sum_{j=1}^n \sum_{i=1}^m p_{ij}$ (the sum of all processing times). Time is discretized into unit periods $t \in \mathcal{T} = \{0, 1, \dots, H\}$.
- L : a sufficiently large constant (big- M), e.g. $L = H$.

We also use the following *derived* sets and quantities, defined purely in terms of the parameters above (they are not independent inputs to the model):

$$M_s(j) := \{i \in M_s : E_{ij} = 1\}, \quad \text{the machines at stage } s \text{ eligible to process job } j,$$

$$W_i := \sum_{j=1}^n p_{ij} x_{ij}, \quad \text{the total workload assigned to machine } i \text{ under allocation } x.$$

By convention, if job j bypasses stage s entirely then $E_{ij} = 1$ for all $i \in M_s$ and $p_{ij} = 0$ for all $i \in M_s$ – bypass and eligibility are independent concepts and must not be conflated: bypass means the job skips the *stage*, whereas $E_{ij} = 0$ means the job must avoid a *specific machine* while still being processed somewhere else at that same stage.

Variable	Domain	Meaning
x_{ij}	$\{0, 1\}$	1 if job j is assigned to machine i
$y_{ijj'}$	$\{0, 1\}$	1 if job j precedes job j' on machine i
C_{ij}	$\mathbb{R}_{\geq 0}$	Completion time of job j on machine i
z_{jst}	$\{0, 1\}$	1 if job j is waiting in the buffer between stage s and $s+1$ at time $t \in \mathcal{T}$
T	$\mathbb{R}_{\geq 0}$	Cycle time (makespan of one MPS)

Decision variables.

2.2 MIP Model

$$\text{minimize } T \tag{1}$$

$$\text{s.t. } \sum_{i \in M_s(j)} x_{ij} = 1 \quad \forall j, \forall s \tag{C1a}$$

$$x_{ij} \leq E_{ij} \quad \forall i, \forall j \tag{C1b}$$

$$C_{ij} \geq C_{ij'} + p_{ij}x_{ij} - L(1 - y_{ijj'}) \quad \forall i, \forall j \neq j' \tag{C2a}$$

$$C_{ij'} \geq C_{ij} + p_{ij'}x_{ij'} - L y_{ijj'} \quad \forall i, \forall j \neq j' \tag{C2b}$$

$$C_{ij} \geq C_{i'j} + p_{ij}x_{ij} \quad \forall j, s, i \in M_s, i' \in M_{s-1} \tag{C3}$$

$$z_{jst} \geq x_{i'j} - \frac{C_{i'j} - t}{L} - \frac{t - C_{ij} + 1}{L} \quad \forall j, s, i' \in M_s, i \in M_{s+1}, t \in \mathcal{T} \tag{C4a}$$

$$\sum_{j=1}^n z_{jst} \leq b_s \quad \forall s, \forall t \in \mathcal{T} \tag{C4b}$$

$$T \geq C_{ij} \quad \forall i, j : x_{ij} = 1 \tag{C5}$$

$$T \geq W_i = \sum_{j=1}^n p_{ij} x_{ij} \quad \forall i \tag{C6}$$

$$x_{ij}, y_{ijj'}, z_{jst} \in \{0, 1\}, \quad C_{ij}, T \geq 0 \tag{C7}$$

(C1a) assigns every job to exactly one *eligible* machine per stage – restricting the summation to $M_s(j)$ rather than the full stage M_s already excludes ineligible machines from the feasible region, and bypassed stages satisfy it trivially since $p_{ij} = 0$ for all $i \in M_s(j)$. (C1b) is an explicit, redundant-but-clarifying restatement of the same restriction directly on x_{ij} : it forbids assigning job j to any machine i with $E_{ij} = 0$, regardless of processing time. We retain both forms because (C1a) is the form a solver actually branches on (a smaller, pre-filtered sum), while (C1b) is the form that should be used if the model is generated programmatically with x_{ij} ranging over *all* $i \in M_s$ for convenience – in that case (C1a) would need to be replaced by $\sum_{i \in M_s} x_{ij} = 1$ and (C1b) becomes the constraint doing the actual exclusion work. Without *either* form, a solver could legally satisfy machine assignment by placing a zero-duration “ghost” visit on an ineligible machine, which is a materially different (and incorrect) relaxation of the problem from bypass, where a zero-duration visit is genuinely correct because the job truly skips that stage. (C2a–C2b) are the classical disjunctive constraints preventing two jobs from overlapping on the same machine. (C3) enforces the series structure of the flow line: a job cannot start processing at stage s before completing stage $s - 1$.

(C4a–C4b) jointly enforce the buffer capacity and are now fully linear. The auxiliary binary z_{jst} indicates whether job j is physically occupying the buffer between stage s and $s + 1$ at discrete time t – that is, whether j has completed processing at its stage- s machine i' (so $C_{i'j} \leq t$) but has not yet started processing at its stage- $(s+1)$ machine i (so $t < C_{ij}$, i.e. $t \leq C_{ij} - 1$ for integer time). Constraint (C4a) is a standard big- L linearization that forces $z_{jst} = 1$ whenever both conditions hold for the assigned machines ($x_{i'j} = x_{ij} = 1$), and allows $z_{jst} = 0$ otherwise; it is linear in all decision variables once t ranges over the finite, explicitly defined index set $\mathcal{T}$. Constraint (C4b) then simply caps, at every discrete time $t \in \mathcal{T}$, the number of jobs simultaneously occupying the buffer between stage s and $s + 1$ at b_s . (C5) defines the cycle time as the latest completion time across all jobs and machines. (C6) is a valid inequality stating that the cycle time cannot be smaller than the total workload W_i of any single machine, where W_i is the derived quantity defined in Section 2.1 – this tightens the LP relaxation considerably and is not an independent constant.

Component	Count
Binary variables x_{ij}	$m \times n$
Binary sequencing variables $y_{ijj'}$	$m \times n \times (n - 1)$
Binary buffer-occupancy variables z_{jst}	$n \times S \times H$
Continuous variables C_{ij}	$m \times n$
Eligibility constraints (C1b)	$O(mn)$
Disjunctive constraints (C2a)–(C2b)	$O(mn^2)$
Buffer constraints (C4a)–(C4b)	$O(n \cdot S \cdot H)$

Table 2: Growth of the MIP’s size with problem dimensions. H is the planning horizon defined in Section 2.1, typically of order $\sum_{ij} p_{ij}$.

2.3 Model Size and Complexity

The eligibility constraints (C1b) add negligible overhead – they are simple variable bounds, not disjunctive or time-indexed, so their count grows only linearly in $m \times n$ and they have no material effect on solver difficulty. Even for a modest instance with $m = 5$ machines and $n = 10$ jobs, the number of binary sequencing variables already reaches 450, and the disjunctive constraints yield a notoriously weak LP relaxation – a well-documented issue for job-shop-style MIPs [4]. The buffer variables z_{jst} are the most problematic component: their count grows with the planning horizon H , which itself grows with total processing time, so doubling typical processing times roughly doubles the number of buffer variables – exploding the model size for any non-trivial instance.

2.4 Critical Assessment of the MIP

Strengths.

- Provides an exact representation; guarantees global optimality given sufficient solver time.
- The bottleneck lower bound (C6) tightens the relaxation and accelerates branch-and-bound.
- Flexible: alternative objectives (WIP, weighted tardiness) can be substituted directly into (1).
- Useful as an exact benchmark for evaluating heuristic solution quality on small instances.
- Machine eligibility (C1b) – the requirement that some jobs be restricted to specific machines within a stage – is captured at negligible cost, since it is a simple variable bound rather than a disjunctive or time-indexed constraint.

Weaknesses.

- The disjunctive constraints (C2a)–(C2b) are the classical weakness of job-shop MIPs, producing enormous branch-and-bound trees even for small instances.
- The buffer constraints (C4a)–(C4b), although now linear, require a time-indexed binary variable for every (job, stage-boundary, time-period) triple – a representation that is exact but whose size scales directly with the planning horizon H .
- The cyclic nature of the schedule – where the end of one MPS cycle feeds directly into the start of the next – is not captured by the static formulation above without additional wrap-around constraints.
- The bypass feature, while trivial to encode via $p_{ij} = 0$, interacts subtly with the buffer constraints, since bypassing jobs still occupy the material handling system.

- The model does not scale: industrial PCB assembly lines may involve dozens of stages and hundreds of job types per MPS, far beyond the practical reach of exact MIP solvers.

Overall verdict. The MIP is theoretically sound and a useful small-instance benchmark, but it is not a practical solution method for industrial-scale instances of this problem – precisely the motivation behind IBM’s development of the FFL heuristic [2].

3 The FFL Algorithm

3.1 Algorithm Description

The Flexible Flow Line Loading (FFLL) algorithm [2, 4] solves the problem in three sequential phases, each addressing one of the three core decisions: *who* processes *what*, *in what order*, and *when*.

Phase 1 — Machine Allocation (LPT Heuristic)

Each job is assigned to one machine per stage so that workloads across parallel machines at that stage are as balanced as possible – balance directly determines the bottleneck workload and hence the achievable cycle time. As noted in Section 1.1, not all machines within a stage are necessarily identical, and a job may be restricted to a subset $M_s(j) \subseteq M_s$ of eligible machines at that stage (the same eligibility notion introduced for the MIP in Section 2.1). We therefore apply an *eligibility-restricted* Longest Processing Time (LPT) heuristic [4] independently at every stage with multiple machines:

1. Sort all jobs in decreasing order of processing time at that stage.
2. Assign each job, one at a time, to the currently least-loaded *eligible* machine at that stage, i.e. $\arg \min_{i \in M_s(j)} (\text{current load of } i)$, rather than the least-loaded machine overall.
3. Repeat until all jobs are assigned.

When every job is eligible for every machine at a stage ($M_s(j) = M_s \forall j$), this reduces exactly to standard LPT. When eligibility is restrictive – e.g. only one machine at a stage can run a particular job – the heuristic degenerates to simply assigning that job to its sole eligible machine, regardless of load; balance is then only achievable among whatever jobs retain a genuine choice. This is an unavoidable consequence of eligibility restrictions, not a defect of the heuristic: the same loss of balancing freedom occurs in the exact MIP of Section 2, where eligibility (C1b) restricts the feasible region in exactly the same way before any optimization takes place.

Phase 2 — Sequencing (Dynamic Balancing Heuristic)

Let n be the number of jobs and m the number of machines. Define

$$\begin{aligned}
 W_i &= \sum_{j=1}^n p_{ij} && \text{workload of machine } i, \\
 W &= \sum_{i=1}^m W_i && \text{total system workload,} \\
 p_k &= \sum_{i=1}^m p_{ik} && \text{system-wide workload of job } k, \\
 o_{ij} &= p_{ij} - p_j \frac{W_i}{W} && \text{overload contribution of job } j \text{ to machine } i, \\
 O_{ij} &= \sum_{k \in S_j} o_{ik} && \text{cumulative overload on machine } i \text{ after position } j,
 \end{aligned}$$

where S_j is the set of jobs loaded up to and including position j in the sequence. The Dynamic Balancing heuristic greedily minimizes

$$\sum_{i=1}^m \sum_{j=1}^n \max(O_{ij}, 0) \quad (2)$$

by, at each position, selecting from the remaining unscheduled jobs the one that minimizes the resulting total positive cumulative overload.

Phase 3 — Timing of Releases (Bottleneck Anchoring)

1. Identify the bottleneck machine $i^* = \arg \max_i W_i$; the cycle time lower bound is $T^* = W_{i^*}$.
2. Release all jobs into the system as rapidly as possible and simulate the resulting schedule.
3. Fix the start/completion times on the bottleneck machine – no idle time is permitted there.
4. For machines *upstream* of the bottleneck, delay processing as much as possible without altering job sequences, reducing WIP.
5. For machines *downstream* of the bottleneck, process jobs as early as possible, again without altering sequences.

3.2 Three Original Application Examples

The following three examples are constructed independently of the textbook and differ from Pinedo's Example 16.3.1. The first two cover the base FFL mechanics; the third specifically demonstrates machine eligibility restrictions (Section 1.1).

Example A — Two Stages, Four Jobs, No Bypass

Stage 1 has two parallel machines; Stage 2 has a single machine; the MPS contains four jobs.

	Job 1	Job 2	Job 3	Job 4
Stage 1	5	8	3	6
Stage 2	4	2	7	5

Table 3: Stage processing times p'_{kj} for Example A.

Phase 1 (LPT at Stage 1). Sorting descending: Job 2 (8), Job 4 (6), Job 1 (5), Job 3 (3). Assignment: Job 2→M1 (load 8), Job 4→M2 (load 6), Job 1→M2 (load 11), Job 3→M1 (load 11). Both Stage 1 machines balance perfectly at load 11; Machine 3 (Stage 2) processes all jobs, $W_3 = 18$. **Bottleneck: Machine 3, $T^* = 18$.**

Phase 2 (Dynamic Balancing). With $W = (11, 11, 18)$, $\sum W = 40$, and $p_k = (9, 10, 10, 11)$, evaluating o_{ij} and applying the greedy rule position-by-position yields the sequence **1 → 3 → 2 → 4**.

Phase 3 (Timing). The bottleneck machine processes jobs back-to-back in this order; the achieved cycle time is $T = 18$, matching the lower bound exactly (optimal for this instance).

Example B — Three Stages, Five Jobs, with Bypass

Stage 1 has one machine; Stage 2 has two parallel machines; Stage 3 has one machine; the MPS contains five jobs, with Jobs 2 and 5 bypassing Stage 2.

Phase 1 (LPT at Stage 2). Only Jobs 1, 3, 4 require Stage 2. Sorting descending: Job 3 (6), Job 1 (4), Job 4 (3). Assignment: Job 3→M2 (load 6), Job 1→M3 (load 4), Job 4→M3 (load 7). The resulting workloads are $W = (20, 6, 7, 12)$, $\sum W = 45$. **Bottleneck: Machine 1 (Stage 1), $T^* = 20$.**

	Job 1	Job 2	Job 3	Job 4	Job 5
Stage 1	3	5	2	4	6
Stage 2	4	0	6	3	0
Stage 3	2	3	1	4	2

Table 4: Stage processing times p'_{kj} for Example B (Jobs 2, 5 bypass Stage 2).

Phase 2 (Dynamic Balancing). With $p_k = (9, 8, 9, 11, 8)$, evaluating o_{ij} and applying the greedy rule position-by-position yields the sequence $\mathbf{2} \rightarrow \mathbf{1} \rightarrow \mathbf{3} \rightarrow \mathbf{5} \rightarrow \mathbf{4}$.

Phase 3 (Timing). The bottleneck Machine 1 processes jobs back-to-back in this order; Jobs 2 and 5 bypass Stage 2 directly. The achieved cycle time is $T = 20$, again matching the lower bound exactly.

Example C — Two Stages, Six Jobs, with General Machine Eligibility Restrictions

Eligibility restrictions in practice are not limited to a simple "one restricted machine vs. one universal machine" pattern: at a stage with several machines, *each* job can have its own eligible subset, of any size, independent of the other jobs' subsets. This example demonstrates that general case directly: Stage 1 has *four* parallel machines, and the six jobs have eligible-machine counts ranging from 1 of 4 up to 4 of 4. Stage 2 has a single machine, so eligibility there is moot.

	Job 1	Job 2	Job 3	Job 4	Job 5	Job 6
Stage 1	6	5	7	4	8	3
Stage 2	2	3	2	4	3	2

Table 5: Stage processing times p'_{kj} for Example C.

Job	Eligible machines at Stage 1
1	$\{1, 2, 3\}$ (3 of 4)
2	$\{2, 4\}$ (2 of 4)
3	$\{1, 3, 4\}$ (3 of 4)
4	$\{3\}$ (1 of 4)
5	$\{1, 2, 3, 4\}$ (4 of 4, unrestricted)
6	$\{1, 4\}$ (2 of 4)

Table 6: Eligibility sets $M_1(j)$ for Example C, illustrating that restrictions can affect any subset of machines, of any size, not merely a single "restricted machine vs. universal machine" pattern.

Phase 1 (eligibility-restricted LPT at Stage 1). Sorting descending by Stage-1 time: Job 5 (8), Job 3 (7), Job 1 (6), Job 2 (5), Job 4 (4), Job 6 (3). We track the load on each of the four machines as the algorithm proceeds, restricting the "least-loaded" choice at each step to that job's eligible set:

1. **Job 5** (eligible: $\{1, 2, 3, 4\}$, all tied at load 0) $\rightarrow$ Machine 1 (load 0 $\rightarrow$ 8).
2. **Job 3** (eligible: $\{1, 3, 4\}$; loads are 8, $-\infty$, 0, 0 among eligible machines) $\rightarrow$ Machine 3 (load 0 $\rightarrow$ 7).
3. **Job 1** (eligible: $\{1, 2, 3\}$; loads are 8, 0, 7) $\rightarrow$ Machine 2, the least loaded eligible machine (load 0 $\rightarrow$ 6).
4. **Job 2** (eligible: $\{2, 4\}$; loads are 6, 0) $\rightarrow$ Machine 4 (load 0 $\rightarrow$ 5).

5. **Job 4** (eligible: {3} *only*) $\rightarrow$ Machine 3 by necessity (load $7 \rightarrow 11$), even though Machine 3 is already the most heavily loaded eligible machine for Job 1 a moment earlier and is, after this step, the most heavily loaded machine at the stage overall. This is the general case of a job with only one eligible machine out of several: it must be placed there regardless of current load.
6. **Job 6** (eligible: {1, 4}; loads are 8, 5) $\rightarrow$ Machine 4 (load $5 \rightarrow 8$).

The resulting Stage-1 workloads are $W_{1:4} = (8, 6, 11, 8)$. Job 4's forced placement on Machine 3 – the single machine it is eligible for – is what pushes Machine 3 to 11, well above the other three machines; no amount of cleverness in ordering the other jobs could have prevented this, since Job 4 has no alternative. Including Stage 2 (a single machine processing all six jobs, total load 16), the full machine workload vector is $W = (8, 6, 11, 8, 16)$, $\sum W = 49$. **Bottleneck: Machine 5 (Stage 2)**, $T^* = 16$.

Contrast with the unrestricted case. If eligibility were ignored entirely (every job eligible for every Stage-1 machine), LPT would instead produce $W_{1:4} = (8, 7, 9, 9)$ – a range of just 2 (from 7 to 9) versus the range of 5 (from 6 to 11) obtained under the eligibility restriction above. The overall bottleneck is unaffected here (Machine 5 at Stage 2 dominates either way, since its load of 16 exceeds any Stage-1 machine's load in both cases), but Stage 1's *internal* balance is visibly worse under the restriction – exactly the effect described in Section 3.1, now shown with eligible-set sizes other than the trivial 1-of-2 case.

Phase 2 (Dynamic Balancing). Evaluating o_{ij} and applying the greedy rule position-by-position yields the sequence **6 $\rightarrow$ 4 $\rightarrow$ 1 $\rightarrow$ 5 $\rightarrow$ 3 $\rightarrow$ 2**.

Phase 3 (Timing). The bottleneck Machine 5 processes jobs back-to-back in this order; the achieved makespan is 19.00, giving a gap of 18.8% to the lower bound $T^* = 16$.

3.3 The Profile Fitting (PF) Heuristic and Comparison with FFL

As requested, we apply a second sequencing heuristic to the same problem and compare it against FFL: **Profile Fitting (PF)**, originally proposed by McCormick et al. [3] for cyclic sequencing in zero-buffer (blocking) assembly lines, and presented by Pinedo immediately before Section 16.3 as a heuristic for the simpler zero-buffer case. Pinedo's own worked example there runs *both* an unweighted and a bottleneck-weighted variant of PF on the same instance and shows the weighted variant achieving a strictly shorter cycle time (12 versus 14). We therefore implement and compare *both* variants here, adapted to the finite-buffer environment of Section 16.3, so that all three sequencing rules – Dynamic Balancing, plain PF, and weighted PF – can be compared on identical instances.

Algorithm Description

PF builds the job sequence one position at a time, exactly like Dynamic Balancing, but with a different greedy criterion: instead of minimizing cumulative *overload* (2), it minimizes cumulative *nonproductive time* – the sum of machine idle time and machine blocked time.

1. **Select the first job.** Choose the job with the largest total processing time across all machines (the variant Pinedo describes as "largest amount of processing").
2. **Generate the profile.** Simulate the partial sequence built so far, recording, for every machine, the idle time (machine waiting for a job to arrive) and blocked time (a finished job cannot advance because the immediately downstream buffer is full).
3. **Evaluate candidates.** For every unscheduled job, tentatively append it to the sequence and recompute the resulting weighted sum of idle and blocked time across all machines.
4. **Select** the candidate job minimizing this sum.

5. **Repeat** steps 2–4 until all jobs are sequenced.

Step 3 requires a per-machine weight vector, and Pinedo’s chapter presents *two* concrete choices for it, both of which we implement:

- **Unweighted PF.** Every machine’s nonproductive time counts equally (weight 1 on every machine). This is the baseline variant.
- **Weighted PF (PF_w).** Nonproductive time on the bottleneck machine is weighted more heavily than on the rest – Pinedo’s own example weights the two constraining machines at 1 and the non-bottleneck machine at 0, in the spirit of the Optimized Production Technology (OPT) philosophy that bottleneck delay is more costly than non-bottleneck delay. We adopt this same weighting: weight 1 on the bottleneck machine identified after Phase 1, and weight 0 elsewhere.

Adaptation to finite buffers. In Pinedo’s original (zero-buffer) setting, a job is blocked the instant it finishes and the next machine is occupied, since there is nowhere to put it. In our finite-buffer setting, a job is blocked only once the buffer immediately downstream has reached its capacity b_s – the direct finite-buffer analogue of the original blocking condition. With $b_s \rightarrow 0$ for all s , this adapted PF coincides with the original zero-buffer PF.

Comparison with FFL on Examples A and B

Applying both PF variants (with buffer capacity $b_s = 2$) to the two examples of Section 3.2 gives:

	Sequence	Makespan	Gap to LB
<i>Example A ($LB = 18$)</i>			
FFL (Dynamic Balancing)	$1 \rightarrow 3 \rightarrow 2 \rightarrow 4$	23.00	27.8%
PF (unweighted)	$4 \rightarrow 1 \rightarrow 2 \rightarrow 3$	24.00	33.3%
PF_w (weighted)	$4 \rightarrow 1 \rightarrow 2 \rightarrow 3$	24.00	33.3%
<i>Example B ($LB = 20$)</i>			
FFL (Dynamic Balancing)	$2 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 4$	27.00	35.0%
PF (unweighted)	$4 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 3$	27.00	35.0%
PF_w (weighted)	$4 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 5$	23.00	15.0%

Table 7: FFL vs. both PF variants on Examples A and B, evaluated under identical Phase 1 allocation and Phase 3 timing (only the sequencing phase differs). Makespan is the simulated makespan of one MPS pass under a finite buffer of capacity 2 per stage.

On Example A, the two PF variants happen to coincide: Stage 1’s two machines are already perfectly balanced by LPT ($W_1 = W_2 = 11$), so weighting the bottleneck machine alone versus weighting all three machines equally does not change which candidate job minimizes nonproductive time at any position. Example B breaks this tie and reproduces, in miniature, exactly the effect Pinedo’s own example demonstrates: weighting the bottleneck machine changes the selected sequence and yields a markedly better result (23.00 vs. 27.00, a 14.8% reduction) than the unweighted variant. Neither PF variant nor FFL dominates throughout: FFL sequences Example A best, while PF_w sequences Example B best. This motivates testing all three on a larger, more systematic sample.

Comparison with FFL on Ten Random Instances

We re-use the same ten randomly generated instances and generation procedure as in Section 4.3, applying both PF variants with the same Phase 1 allocation and Phase 3 timing as FFL, so that the comparison isolates the effect of the sequencing phase alone.

#	Jobs	Stg	LB	FFLL	FFLL gap	PF	PF gap	PF _w	PF _w gap
1	4	2	9.0	13.00	44.4%	13.00	44.4%	12.00	33.3%
2	5	2	23.0	26.00	13.0%	32.00	39.1%	32.00	39.1%
3	6	3	32.0	35.00	9.4%	40.00	25.0%	40.00	25.0%
4	7	3	38.0	42.00	10.5%	51.00	34.2%	51.00	34.2%
5	8	4	21.0	43.00	104.8%	47.00	123.8%	48.00	128.6%
6	8	3	34.0	49.00	44.1%	55.00	61.8%	47.00	38.2%
7	10	4	46.0	56.00	21.7%	73.00	58.7%	61.00	32.6%
8	10	5	49.0	56.00	14.3%	65.00	32.7%	63.00	28.6%
9	12	4	66.0	97.00	47.0%	89.00	34.8%	85.00	28.8%
10	12	5	63.0	81.00	28.6%	92.00	46.0%	90.00	42.9%
Average				33.8%		50.1%		43.1%	

Table 8: FFLL vs. both PF variants on the ten random instances of Section 4.3. Gaps are relative to the bottleneck lower bound (LB).

Three findings emerge. **First**, and most directly relevant to Pinedo’s own claim, PF_w improves on plain PF in 6 of the 10 instances (and ties on the remaining 4, where the same balanced-machine coincidence as Example A occurs) – the bottleneck-weighting refinement is, on average, beneficial, exactly as the textbook’s worked example illustrates. The average gap falls from 50.1% (unweighted PF) to 43.1% (weighted PF_w). **Second**, FFLL’s Dynamic Balancing outperforms both PF variants on average in our finite-buffer adaptation, with the lowest mean gap (33.8%) and the best result on 7 of the 10 instances (versus 3 for PF_w and 0 for plain PF outright, since PF_w is never strictly worse than PF in our runs). **Third**, the gap between FFLL and PF widens on the larger instances (7–10), suggesting Dynamic Balancing’s forward-looking overload measure (2) scales better than PF’s nonproductive-time measure as problem size grows. A plausible explanation is that Dynamic Balancing accounts for workload imbalance *before* it materializes as idle or blocked time, whereas PF only reacts to nonproductive time *after* a tentative placement – a less forward-looking signal that may compound over longer sequences. This is a meaningful empirical finding given that PF was originally designed for the zero-buffer case, not the finite-buffer case studied here.

3.4 Critical Assessment of the FFLL Heuristic

Strengths.

- Computationally efficient: Phase 1 runs in $O(n \log n)$ per stage, Phase 2 in $O(n^2 m)$, and Phase 3 in linear time – tractable for the hundreds of job types found in industrial MPS instances.
- The three-phase decomposition mirrors the natural decision hierarchy: allocation must precede sequencing (workloads must be known first), and sequencing must precede timing.
- The Dynamic Balancing objective (2) has a clear theoretical motivation: minimizing burst loading directly reduces buffer occupancy and blocking probability.
- Bypass is handled naturally, since bypassed stages simply contribute zero to all o_{ij} terms.
- Machine eligibility restrictions are handled by a straightforward modification of LPT (Section 3.1) that restricts each job’s candidate machine to its eligible subset $M_s(j)$, at no extra computational cost.
- Extensive practical experiments reported by Pinedo confirm FFLL’s value as a scheduling tool in real assembly environments [4].

Limitations.

- The three phases are solved *independently and sequentially* with no feedback loop – a poor Phase 1 allocation cannot be corrected by Phases 2 or 3.
- LPT does not guarantee optimal balance; Pinedo himself notes this explicitly for Stage 3 of Example 16.3.1.
- Severe eligibility restrictions can defeat balancing entirely: if only one machine at a stage is eligible for a given job, that job must go there regardless of current load, and the achievable balance is limited to whatever flexibility remains among jobs with a genuine choice of machine.
- The Dynamic Balancing heuristic is *greedy with no lookahead*: a locally optimal choice at position j may preclude a substantially better global sequence.
- WIP minimization is addressed only indirectly, through Phase 3's timing adjustments, with no explicit guarantee against blocking under tight buffers.
- The algorithm assumes a fixed, fully known MPS and offers no mechanism for incremental re-optimization under dynamic arrivals or uncertain processing times.
- Although Dynamic Balancing outperformed both adapted PF variants on average (Section 3.3), PF_w still won on 3 of 10 random instances and on Example B, showing that no single greedy sequencing rule dominates across all instances – a further argument for the iterated, non-greedy approach developed in Section 4.

4 Improved Algorithm: Iterated Greedy with Local Search (IG-LS)

4.1 Design Rationale

The principal weakness identified in Section 3.4 is that FFLL's Phase 2 is a *non-backtracking greedy* procedure: once a job is placed, it remains fixed regardless of how later choices unfold. This is precisely the failure mode that the *iterated greedy* (IG) metaheuristic framework of Ruiz and Stützle [5] was designed to address for permutation flow shop scheduling, using a simple destruction–reconstruction cycle that the original paper itself shows is competitive with, and often superior to, considerably more elaborate metaheuristics while remaining straightforward to implement [5]. This motivates adapting the same destruction–reconstruction principle to the FFLL sequencing phase.

We propose **IG-LS**, which retains FFLL's Phase 1 (LPT allocation) and Phase 3 (bottleneck timing) unchanged, and replaces Phase 2 with three layers:

1. **Warm start.** Initialize with the FFLL Dynamic Balancing sequence.
2. **Local search (2-opt).** Exhaustively evaluate pairwise job swaps; accept any swap that reduces the *actual simulated makespan*; repeat until no improving swap remains.
3. **Iterated perturbation.** Remove a random contiguous block of jobs and reinsert it at a random position (destruction–reconstruction, following Ruiz and Stützle [5]); re-apply local search; accept the new solution only if it improves the incumbent makespan. Repeat for a fixed iteration budget.

Crucially, IG-LS optimizes the *true simulated makespan* rather than the overload proxy (2) used by Dynamic Balancing – a more direct and unbiased optimization signal.

Algorithm 1 IG-LS: Iterated Greedy with Local Search

```

1: procedure IG-LS(instance, n_iter = 150, block_size = 2)
2:   alloc  $\leftarrow$  LPT_Allocate(instance)  $\triangleright$  Phase 1, identical to FFL
3:   seq  $\leftarrow$  DynamicBalancing(alloc)  $\triangleright$  warm start
4:   (seq, best_ms)  $\leftarrow$  LocalSearch2opt(seq, alloc)
5:   for  $k = 1$  to n_iter do
6:     seq'  $\leftarrow$  Perturb(seq, block_size)  $\triangleright$  remove & reinsert block
7:     (seq', ms')  $\leftarrow$  LocalSearch2opt(seq', alloc)
8:     if ms' < best_ms then
9:       seq  $\leftarrow$  seq'; best_ms  $\leftarrow$  ms'
10:    end if
11:  end for
12:  cycle_time  $\leftarrow$  BottleneckTiming(seq, alloc)  $\triangleright$  Phase 3, identical to FFL
13:  return seq, cycle_time, best_ms
14: end procedure

```

4.2 Pseudocode

4.3 Experimental Results on Ten Random Instances

Ten instances were generated with deliberately varied characteristics – ranging from small 4-job/2-stage problems to larger 12-job/5-stage problems, with varying bypass probabilities and numbers of machines per stage (full generator and parameters in the accompanying source code, `ffll_igls.py`). The lower bound (LB) reported is the bottleneck machine workload after LPT allocation (constraint (C6)).

#	Jobs	Stg	LB	FFLL	FFLL gap	IG-LS	IG-LS gap	Improv.
1	4	2	9.0	13.00	44.4%	12.00	33.3%	7.7%
2	5	2	23.0	26.00	13.0%	25.00	8.7%	3.8%
3	6	3	32.0	35.00	9.4%	32.00	0.0%	8.6%
4	7	3	38.0	42.00	10.5%	38.00	0.0%	9.5%
5	8	4	21.0	43.00	104.8%	34.00	61.9%	20.9%
6	8	3	34.0	49.00	44.1%	35.00	2.9%	28.6%
7	10	4	46.0	56.00	21.7%	50.00	8.7%	10.7%
8	10	5	49.0	56.00	14.3%	54.00	10.2%	3.6%
9	12	4	66.0	97.00	47.0%	72.00	9.1%	25.8%
10	12	5	63.0	81.00	28.6%	73.00	15.9%	9.9%
Average					33.8%		15.1%	12.9%

Table 9: FFLL vs. IG-LS on ten randomly generated instances. Gaps are relative to the bottleneck lower bound (LB). Bold entries indicate proven-optimal solutions (gap = 0%).

IG-LS improved *every one* of the ten instances. On Instances 3 and 4 it reached the theoretical lower bound, proving optimality. The largest gains occur on Instances 5, 6, and 9, where FFLL's greedy sequencing committed to particularly poor early choices that IG-LS's perturbation and local search were able to correct. Across all ten instances, the average gap to the lower bound fell from 33.8% (FFLL) to 15.1% (IG-LS), while all runs completed in well under one second.

4.4 Critical Assessment of IG-LS

Strengths over FFLL.

- More than halves the average optimality gap (33.8% $\rightarrow$ 15.1%).
- Reaches proven optimality on two of ten instances.

- Improves consistently across all tested instance types, not just isolated cases.
- Optimizes actual makespan directly, rather than an overload proxy.
- Remains computationally cheap (sub-second per instance at this scale), consistent with reports in the IG literature that the framework is simple yet highly competitive against more elaborate metaheuristics [5].

Remaining limitations.

- The gap to the lower bound remains substantial on some instances (e.g., Instance 5: 61.9%); note the bound itself may be loose, so the true optimality gap could be smaller than reported.
- The purely greedy acceptance criterion (accept only strict improvements) may miss solutions reachable only via temporary uphill moves; a simulated-annealing-style acceptance criterion, as used in several IG variants [5], could help.
- The perturbation block size is fixed at 2; an adaptive schedule could better balance exploration and exploitation.
- Like FFLL, the three phases remain solved independently; a fully integrated metaheuristic spanning allocation and sequencing could yield further gains at higher computational cost.
- Scalability beyond the tested range (up to 12 jobs) is untested; the $O(n^2)$ per-iteration cost of 2-opt local search will eventually dominate for much larger MPS instances.

The complete Python implementation (FFLL, IG-LS, instance generator, and experiment runner) accompanies this report as `ffll_igls.py`.

References

- [1] Nicholas G. Hall and Chelliah Sriskandarajah. A survey of machine scheduling problems with blocking and no-wait in process. *Operations Research*, 44(3):510–525, 1996. doi: 10.1287/opre.44.3.510. URL <https://pubsonline.informs.org/doi/abs/10.1287/opre.44.3.510>.
- [2] IBM Corporation. The Flexible Flow Line Loading (FFLL) algorithm for printed circuit board assembly scheduling, 1990. URL <https://doi.org/10.1007/978-3-319-26580-3>. As described in Pinedo, *Scheduling: Theory, Algorithms, and Systems*, Section 16.3.
- [3] S. Thomas McCormick, Michael L. Pinedo, Scott Shenker, and Barry Wolf. Sequencing in an assembly line with blocking to minimize cycle time. *Operations Research*, 37(6):925–935, 1989. doi: 10.1287/opre.37.6.925. URL <https://pubsonline.informs.org/doi/10.1287/opre.37.6.925>.
- [4] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, New York, 5th edition, 2016. doi: 10.1007/978-3-319-26580-3. URL <https://doi.org/10.1007/978-3-319-26580-3>.
- [5] Rubén Ruiz and Thomas Stützle. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3):2033–2049, 2007. doi: 10.1016/j.ejor.2005.12.009. URL <https://www.sciencedirect.com/science/article/abs/pii/S0377221705008507>.